1.3. Identificación y caracterización de los principales lenguajes relacionados con la programación de clientes Web (Criterio 1.3)

- **Resultado de Aprendizaje Asociado: RA1** (*Selecciona las arquitecturas y tecnologías de programación sobre clientes Web, identificando y analizando las capacidades y características de cada una*).

- **Criterio Curricular Oficial: CE 1.c** (Se han identificado y caracterizado los principales lenguajes relacionados con la programación de clientes web).

- **Ponderación del Criterio:** (**16,67% del RA1** | **0,833% sobre la calificación final del módulo**).

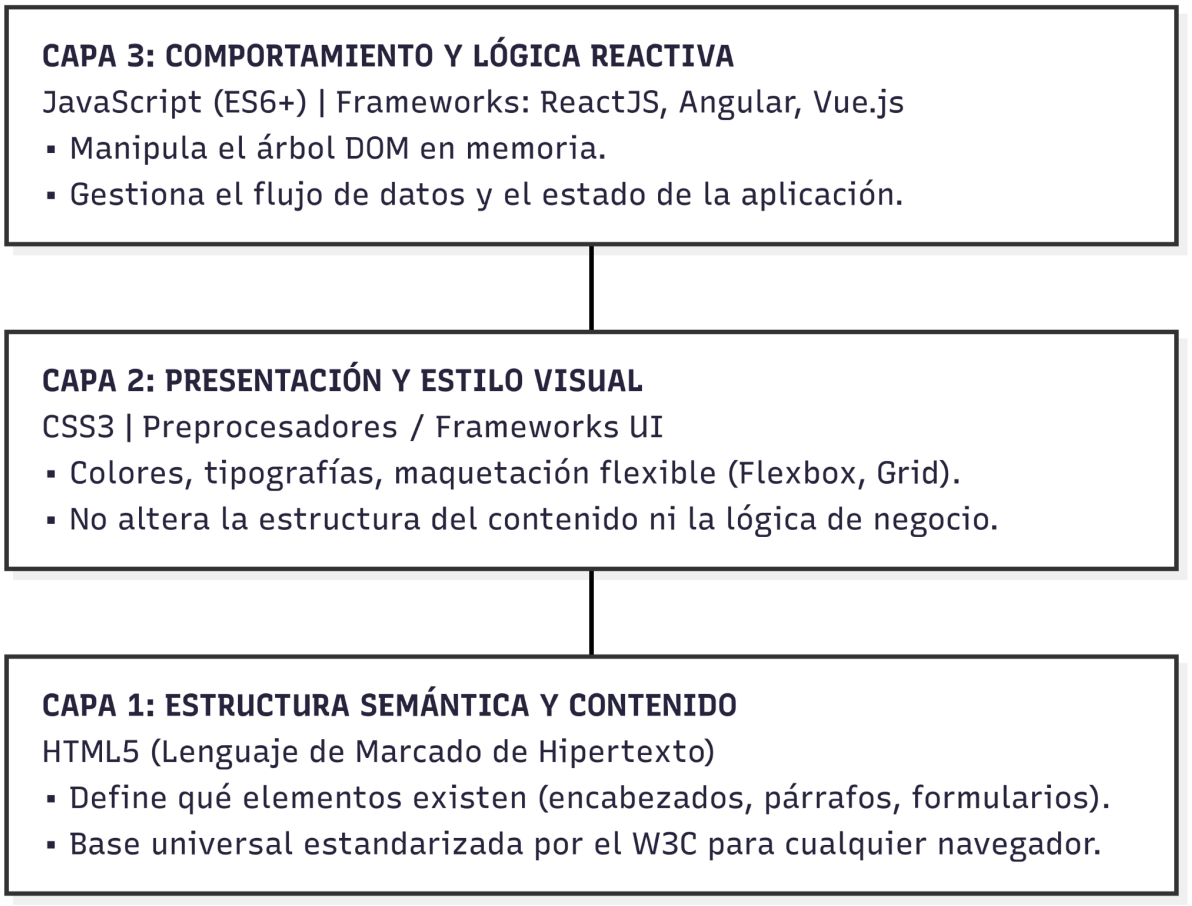
La programación en el entorno del cliente no se reduce a escribir líneas de código aisladas; se fundamenta en una tríada estandarizada y en un ecosistema de librerías y marcos de trabajo (*frameworks*) que estructuran, decoran y dotan de comportamiento reactivo a las aplicaciones web modernas.

A. La Tríada Fundamental de la Programación Cliente

Cualquier página web accesible en internet se apoya de forma directa sobre tres lenguajes estándar, cada uno con una responsabilidad independiente:

- **1. El Lenguaje HTML (HyperText Markup Language):**
    - **Naturaleza:** No es un lenguaje de programación, sino un lenguaje de marcado basado en etiquetas.
    - **Responsabilidad:** Delimita y define la semántica y la estructura del documento (textos, listas, tablas, campos de formulario, botones, imágenes).
    - **Interpretación:** El navegador web lee e interpreta las etiquetas para representarlas de forma visual. Su ventaja reside en que cualquier navegador compatible con los estándares del W3C interpreta la estructura de modo homogéneo.
- **2. El Lenguaje CSS (Cascading Style Sheets):**
    - **Naturaleza:** Lenguaje declarativo de diseño gráfico y maquetación visual.
    - **Responsabilidad:** Define el aspecto, estética, proporciones, márgenes y adaptabilidad a diferentes pantallas (*responsive design*).
    - **Separación de facetas:** No interviene en la lógica ni en los datos de la web; se limita a que la presentación sea atractiva e intuitiva para el usuario.
- **3. El Lenguaje JavaScript:**
    - **Naturaleza:** Auténtico lenguaje de programación dinámico, débilmente tipado y orientado a eventos.
    - **Responsabilidad:** Inyecta dinamismo, reacciona a las acciones del usuario (pulsaciones de teclas, clics, movimientos del ratón), valida entradas y altera la estructura del documento en caliente sin recargar la página.

- Debido a los riesgos del dinamismo extremo en aplicaciones complejas (especialmente financieras), el estándar actual de la industria es utilizar **TypeScript**.
- TypeScript no es un lenguaje nuevo; es una capa sobre JavaScript que le añade **tipado estático**. Escribes código seguro con tipos fijos que se comprueban mientras programas, pero al compilarse se transforma en JavaScript dinámico estándar para que el navegador pueda entenderlo.



B. JavaScript. Evolución histórica

La evolución histórica de JavaScript es una de las más singulares en la historia de la informática: un lenguaje diseñado en solo 10 días para añadir animaciones y validar formularios en páginas estáticas terminó convirtiéndose en el motor de ejecución universal de la web moderna, servidores, aplicaciones de escritorio y dispositivos embebidos.

0. El Origen: 10 días en Netscape (1995)

En mayo de 1995, cuando la web estaba dominada por el navegador Netscape Navigator, nació como un lenguaje de script ligero para navegadores web y ha evolucionado hasta convertirse en la columna vertebral del desarrollo web moderno, tanto en cliente como en servidor.

1. El nacimiento y la estandarización temprana (1995–1999)

- **1995 (Mocha / LiveScript / JavaScript):** Brendan Eich crea el lenguaje en solo 10 días para Netscape Communications. Su objetivo original era dotar de interactividad básica a los documentos HTML (validación de formularios, animaciones sencillas).
- **1996 (JScript):** Microsoft lanza Internet Explorer 3.0 con una versión propia llamada JScript mediante ingeniería inversa, desatando la primera "guerra de navegadores" e introduciendo problemas severos de compatibilidad e interoperabilidad.
- **1997 (ECMAScript 1 - ECMA-262):** Netscape entrega la especificación a Ecma International para fijar un estándar neutro: nace **ECMAScript**.
- **1998 (ES2) y 1999 (ES3):** ES3 consolida el lenguaje durante la siguiente década introduciendo expresiones regulares (RegExp), bloques de manejo de excepciones (try/catch), formateo estricto y mejoras en la manipulación de cadenas y objetos.

2. El estancamiento y la era AJAX (2000–2008)

Tras ES3, el comité técnico (TC39) entró en un largo periodo de desacuerdo:

- **El fracaso de ES4:** Se propuso una reescritura radical con tipado estático, clases complejas y paquetes (muy influenciada por ActionScript 3). Fue abandonada por su excesiva complejidad y la falta de consenso entre Microsoft, Netscape/Mozilla y otros miembros.
- **El auge de AJAX (2005):** Jesse James Garrett acuña el término AJAX. El uso de XMLHttpRequest permite actualizar páginas web sin recargarlas por completo, transformando a JavaScript de un simple adorno a una herramienta de aplicaciones web completas (Google Maps, Gmail).
- **Las librerías de abstracción (2006):** La fragmentación entre navegadores impulsa la creación de librerías como **jQuery**, Prototype y MooTools, cuyo propósito principal era unificar las APIs del DOM y mitigar las inconsistencias entre navegadores.

3. La madurez: ES5 y la salida del navegador (2009)

En 2008, el TC39 acordó abandonar ES4 y avanzar en una propuesta pragmática e incremental conocida como **Harmony**, que dio lugar a:

- **ECMAScript 5 (diciembre de 2009):**
  - Modo estricto ("use strict").
  - Métodos funcionales de arrays (forEach, map, filter, reduce, some, every).
  - Soporte nativo para JSON (JSON.parse, JSON.stringify).
  - Getters y setters, además de control sobre descriptores de propiedades de objetos (Object.defineProperty, Object.freeze, Object.keys).
- **El motor V8 y Node.js (2008–2009):** Google lanza Chrome con el motor **V8** (compilación JIT directa a código máquina), multiplicando el rendimiento. Ryan Dahl crea **Node.js** sobre V8, llevando JavaScript al backend y desencadenando el ecosistema de herramientas basadas en npm.

4. El punto de inflexión: ECMAScript 2015 (ES6)

Publicado en junio de 2015, **ES6** (oficialmente **ECMAScript 2015**) supuso la mayor refundición de la sintaxis y capacidades del lenguaje desde su creación, preparándolo para proyectos de gran escala.

5. La era moderna: Lanzamientos anuales (ES2016+)

A partir de ES6, el TC39 adoptó un proceso de aprobación en 4 fases (*Stages 0 a 4*) con publicaciones anuales para evitar bloqueos y añadir características a medida que maduran:

C. El Ecosistema de Frameworks y Librerías de Front-end

En los entornos profesionales actuales, las aplicaciones raramente se construyen utilizando únicamente JavaScript nativo (*Vanilla JS*), sino apoyándose en un *framework* o librería avanzada.

- **Origen:** Nacieron inicialmente como librerías de funciones destinadas a simplificar tareas repetitivas y solventar las diferencias de implementación entre navegadores.
- **Evolución:** Hoy constituyen plataformas de desarrollo completas que incorporan compilación previa, lenguajes tipados o extensiones de sintaxis como **TypeScript** o **JSX**.
- **Ventajas que aportan los Frameworks al desarrollo empresarial:**
    - **Coste económico nulo:** La gran mayoría son herramientas de código abierto (*open-source*) y de distribución gratuita, eliminando barreras de inversión inicial.
    - **Fiabilidad, seguridad y rendimiento:** Respaldados por comunidades globales y grandes corporaciones, su código base está probado por miles de programadores, minimizando fallos comunes de seguridad y fugas de memoria.
    - **Velocidad de entrega (*Time to Market*):** Incluyen patrones de diseño, componentes prediseñados, gestión de rutas y estructuras comunes ya resueltas.
    - **Estandarización de equipos:** Permiten que nuevos programadores se incorporen a un proyecto en marcha de forma eficiente si ya conocen el estándar de dicho framework.

D. Caracterización de los Principales Frameworks del Mercado

1. ReactJS

- **Origen y soporte:** Librería/framework creado y mantenido por Meta (Facebook), ampliamente adoptado en el mercado mundial.
- **Programación Orientada a Componentes:** La interfaz no se diseña en un bloque monolítico; se divide en piezas independientes y reutilizables llamadas **componentes** (por ejemplo: un botón, una tarjeta de producto, una barra de navegación). Cada componente gestiona su propio **estado (*state*)** interno.
- **El DOM Virtual (*Virtual DOM*):**

    - Manipular el DOM nativo del navegador es una operación lenta porque obliga al motor a recalcular geometrías y repintar píxeles.
    - React mantiene en la memoria RAM una copia ligera del DOM. Cuando los datos cambian, calcula las diferencias mínimas entre el DOM virtual y el real (*reconciliación*), actualizando únicamente los nodos estrictamente necesarios para maximizar el rendimiento.
- **Sintaxis JSX:** Extensión de JavaScript que permite escribir estructuras similares a etiquetas HTML directamente dentro del código de programación, combinando la expresividad del marcado con la potencia del lenguaje.

2. Angular (y el legado de AngularJS)

- **Origen y soporte:** Plataforma de desarrollo desarrollada y mantenida por Google.
- **Evolución:** Su primera versión se llamó **AngularJS** (basada en JavaScript clásico directo). A partir de la versión 2 pasó a denominarse **Angular**, evolucionando hacia una solución integral con una arquitectura más estructurada.
- **Lenguaje base (TypeScript):** Se programa en **TypeScript**, un superconjunto tipado de JavaScript mantenido por Microsoft que añade interfaces, tipado estático y compilación hacia JavaScript estándar.
- **Curva de aprendizaje:** Presenta una curva pronunciada debido a su rigidez estructural, exigiendo dominar inyección de dependencias, TypeScript y programación reactiva con **RxJS**.

3. Vue.js

- **Origen y filosofía:** Diseñado por Evan You con la premisa de tomar las mejores características de React y Angular, priorizando la ligereza y la velocidad de ejecución.
- **Arquitectura:** Emplea también un **DOM virtual** para optimizar el renderizado.
- **Curva de aprendizaje progresiva:** Es mucho más accesible y suave que la de Angular. Suele adoptarse en combinación con frameworks de back-end como **Laravel** para estructurar interfaces interactivas.

4. Otros Frameworks y Librerías Alternativas

El mercado cuenta con diversas alternativas que el técnico debe evaluar según los requisitos de soporte, popularidad y rapidez:

- **EmberJS:** Framework con convención sobre configuración pensado para grandes aplicaciones web empresariales.
- **BackboneJS:** Uno de los primeros intentos de estructurar aplicaciones con modelos y vistas ligeras.
- **MeteorJS:** Plataforma integral de tiempo real que unifica el cliente y el servidor bajo el mismo entorno JavaScript.
- **Aurelia.js, Polymer y Mithril.js:** Alternativas enfocadas en estándares de componentes web (*Web Components*) y motores de renderizado ultra ligeros.

E. Vocabulario Técnico Fundamental del Criterio

- **DOM Virtual (*Virtual DOM*):** Concepto de programación que consiste en que el framework guarde en la memoria RAM una copia del DOM original; sirve para reducir al máximo las renderizaciones en el navegador e incrementar el rendimiento.
- **JSX:** Extensión de JavaScript parecida a un lenguaje de plantillas pero con toda la capacidad de ejecución de JavaScript integrado.
- **TypeScript:** Lenguaje de código abierto (*open-source*) desarrollado y mantenido por Microsoft; actúa como un superconjunto de JavaScript que añade tipado estático para grandes proyectos y se compila a JavaScript ejecutable estándar.
- **Patrón reactivo:** Modelo de programación basado en flujos de datos asíncronos que reacciona de forma automática propagando los cambios en la interfaz cuando el estado de los datos varía.

F. Funciones en JavaScript. Introducción

Una **función** en JavaScript es un bloque de código reutilizable diseñado para realizar una tarea específica. Se define una vez y se puede ejecutar (*invocar*) tantas veces como sea necesario.

1. Formas principales de declararlas

A. Declaración tradicional (*Function Declaration*)

Es la forma clásica. Disfruta de *hoisting* (se puede invocar antes de la línea donde está escrita).

```
function saludar(nombre) {
  return `Hola, ${nombre}`;
}

console.log(saludar("Ana")); // "Hola, Ana"
```

B. Expresión de función (*Function Expression*)

Se asigna una función (normalmente anónima) a una variable o constante. No se puede usar antes de definirla.

```
const duplicar = function(numero) {
  return numero * 2;
};

console.log(duplicar(5)); // 10
```

C. Funciones flecha (*Arrow Functions* - ES6)

Sintaxis moderna y compacta. Si el cuerpo tiene una sola línea, el `return` y las llaves `{}` son implícitos.

```
// Retorno implícito
const sumar = (a, b) => a + b;

// Si recibe un solo parámetro, se pueden omitir los paréntesis
const cuadrado = x => x * x;

console.log(sumar(3, 4)); // 7
```

Ejemplo: Modificar el Contenido de la Página Web

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Modificando HTML con función</title>
</head>
<body>
  <h1>Modificando el código HTML</h1>
  <p id="prueba">Modificando el contenido.</p>

  <button type="button" onclick="cambiarTexto()">
    ¡Dale!
  </button>

  <script>
    function cambiarTexto() {
      document.getElementById('prueba').innerHTML = 'CAMBIANDO el contenido!';
    }
  </script>
</body>
</html>
```

G. Actividades Prácticas y de Consolidación

- **Realiza los ejercicios del apartado anterior haciendo uso de funciones.**
- **Actividad Propuesta 1.1 (del libro):** Averigua qué es la programación reactiva.
    - Investiga cómo se comporta una hoja de cálculo cuando modificas una celda y las celdas dependientes se recalculan de inmediato.
- **Actividad de Análisis Comparativo:**
    - Elabora una tabla justificando qué framework (ReactJS, Angular o Vue.js) elegirías para:
        1. Una pequeña tienda de barrio con presupuesto reducido y necesidad de despliegue rápido.

2. El portal bancario de una entidad financiera con cientos de programadores y exigencias estrictas de tipado robusto.
3. Una aplicación interactiva que requiera renderizado ultra rápido de miles de productos con cambios constantes en pantalla.